SCHOOL OF TECHNOLOGY	

DATABASE MANAGEMENT SYSTEMS

PROJECT-BASED LEARNING (PBL) REPORT

Small Database Application Development using Python

PROJECT TITLE

AttendEase — Student Attendance & Leave Approval Management System

Program	B,Tech
Section	AIML Rhinos
Faculty	Prof. Sahithi,
Date of Submission	14-10-2026
Academic Year	2025– 2029

Submitted by

Roll Number	Student Name	Mobile	Email ID
25WU0102345	Atharva Singh	8104172370	niranjan.a_2029@woxsen.edu.in

ACKNOWLEDGEMENT

I express my sincere gratitude to Woxsen University, School of Technology, for providing me with the opportunity to undertake this Project-Based Learning (PBL) activity as part of the Database Management Systems (DBMS) course.

I thank my course faculty Prof. Sahithi, for the guidance, suggestions, and support throughout the design and development of this project. The guidance helped me understand the practical application of database concepts such as ER modelling, relational schema design, normalization, SQL, constraints, queries, and database application development.

This PBL activity has strengthened my understanding of DBMS concepts and provided valuable experience in applying them to a real-world problem.

UNDERTAKING

I hereby declare that the project entitled "AttendEase — Student Attendance & Leave Approval Management System", submitted as part of the Project-Based Learning activity for the Database Management Systems (DBMS) course, is my individual work carried out under the guidance of the course faculty.

I undertake that the project report, database design, SQL scripts, application source code, results, and supporting materials submitted by me have been prepared for academic purposes. Wherever information, ideas, tools, datasets, or other resources have been referred to, appropriate acknowledgement has been provided.

I further undertake that I am responsible for the correctness and authenticity of the work submitted and that I have followed the academic and project guidelines prescribed by the University.

Date	: 14-10-2026	Name	: Atharva Singh
Place	: Woxsen University	Roll Number	: 25WU0102345
		Program	: B.Tech AIML
		Signature	: _____

TABLE OF CONTENTS

Contents	Page No.
Abstract	8
LIST OF TABLES	5
LIST OF FIGURES	6
LIST OF ACRONYMS	7
1. PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM	9
1.1 Problem Statement	9
1.2 Objectives	9
1.3 Scope and Assumptions	9
1.4 Proposed Solution Strategy	10
1.5 Functional Requirements	10
1.6 Entity-Relationship Diagram (ERD)	11
1.7 Entity and Relationship Summary	13
2. LOGICAL DATABASE DESIGN	14
2.1 Relational Schema	14
2.2 Keys and Integrity Constraints	15
2.3 Functional Dependencies	19
2.4 Normalization up to Third Normal Form (3NF)	20
2.5 Data Dictionary	21
2.6 Implementation of Logical Design: DDL and Sample Data	27
3. APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN	30
3.1 Application Overview	30
3.2 Development Environment and Database Connectivity	30
3.3 Application Modules and Database Mapping	31
3.4 CRUD Operations	32
3.5 Business Rules, Validation and Error Handling	32
3.6 Reports and Database Operations	33
3.7 Application Screens	35
3.8 Testing and Results	36
4. CONCLUSION	38
4.1 Limitations and Future Enhancements	38
5. REFERENCES	39
6. APPENDICES	40

LIST OF TABLES

Table No.	Table Title	Page No.
Table 1	Scope and Assumptions	10
Table 2	Proposed Solution Strategy	10
Table 3	Functional Requirements	11
Table 4	Entity and Relationship Summary	13
Table 5	Relational Schema	14
Table 6	Keys and Integrity Constraints	15
Table 7	Normalization Status up to 3NF	20
Table 8	Data Dictionary	21
Table 9	Database Implementation Environment	27
Table 10	Application Development Environment	30
Table 11	Application Module-to-Database Mapping	31
Table 12	CRUD Operations	32
Table 13	Business Rules, Validation and Error Handling	32
Table 14	Reports and Database Operations	34
Table 15	Test Cases	37

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Figure 1	Entity-Relationship Diagram of the Proposed System	12
Figure 2	Attendance Marking Screen (Data Entry and Update Transaction)	35
Figure 3	Leave Application and Review Screen (Approval Transaction)	35
Figure 4	Student Attendance Summary with Shortage Alert (Search / Display)	36
Figure 5	Reports Hub with CSV Export (Report Output)	36

LIST OF ACRONYMS

Acronym	Expansion
1NF	First Normal Form
2NF	Second Normal Form
3NF	Third Normal Form
ACID	Atomicity, Consistency, Isolation, Durability
BCNF	Boyce–Codd Normal Form
CRUD	Create, Read, Update, Delete
CSS	Cascading Style Sheets
CSV	Comma-Separated Values
DBMS	Database Management System
DDL	Data Definition Language
DML	Data Manipulation Language
ER	Entity-Relationship
ERD	Entity-Relationship Diagram
FD	Functional Dependency
FK	Foreign Key
HTML	HyperText Markup Language
PBL	Project-Based Learning
PK	Primary Key
RBAC	Role-Based Access Control
SQL	Structured Query Language

ABSTRACT

Colleges must record class-by-class attendance, enforce a minimum attendance requirement, and handle student leave requests, yet these activities are often kept in separate registers or spreadsheets that are difficult to reconcile and audit. A database-backed system is needed so that attendance and leave information is stored once, validated consistently, and available to every authorised user. The objective of this project is to design and implement a relational database application that records attendance, applies the attendance policy, and processes leave approvals. The system, named AttendEase, was modelled with an entity-relationship diagram and converted into a normalized relational schema of seventeen tables and two views in MySQL 8.0, using primary keys, foreign keys, unique constraints and a check constraint. The application is written in Python with the Flask framework and provides separate workspaces for students, faculty and administrators. It supports class-session creation, three-state attendance marking, leave submission and review, makeup sessions, automatic shortage alerts, eight SQL reports with CSV export, and an audit trail. Related updates run as transactions with rollback on failure. The prototype demonstrates how database design principles translate into a working application that gives faculty and students one consistent source of attendance and leave information.

KEYWORDS

Keywords: Relational Database, ER Diagram, Normalization, SQL, Python Flask, Attendance and Leave Management

CHAPTER 1
PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

In a college, each class meeting must be recorded against every enrolled student, the resulting attendance percentage must be compared with a minimum requirement, and any absence that is supported by a leave request must be reviewed and traced back to a decision. When these tasks are performed through separate paper registers, spreadsheets or messages, the same fact is stored in several places. Students cannot easily see their own standing in each course, faculty cannot easily tell which absences are covered by an approved leave, and administrators cannot easily produce institution-wide shortage lists or audit who approved what.

The affected stakeholders are students, who need timely visibility of attendance and a clear way to apply for leave; faculty, who need to mark attendance and decide on leave requests; and administrators, who maintain departments, courses, sections and enrolments and need consolidated reports. A relational database is needed so that attendance, enrolment and leave records are stored once, related through keys, protected by integrity constraints, and queried consistently for every role.

1.2 Objectives

- Design an entity-relationship model and a normalized relational schema that captures departments, users, students, faculty, courses, sections, enrolments, class sessions, attendance, leave applications and approvals, makeup sessions, alerts and audit records.
- Develop a Flask web application, backed by MySQL, with role-based workspaces for students, faculty and administrators.
- Implement attendance marking in three states (present, absent, leave) and automate the calculation of attendance percentage and shortage alerts against a configurable threshold (default 75%).
- Implement a leave workflow in which a student submits a request and a reviewer approves or rejects it, with the decision recorded in an audit table within a single transaction.
- Generate eight analytical SQL reports, each exportable to CSV, using joins, grouping, aggregate functions, a view and a derived-table subquery.

1.3 Scope and Assumptions

Table 1 states what the prototype covers and the assumptions under which it operates.

Table 1. Scope and Assumptions

In Scope	Role-based login (administrator, faculty, student); management of departments, students, faculty, courses, sections and enrolments; class-session creation and attendance marking; attendance percentage and shortage alerts; leave application with optional document upload, review and decision audit; makeup sessions and their attendance; eight SQL reports with CSV export; dashboards with Chart.js charts; audit logging.
Out of Scope / Exclusions	Timetable generation; biometric, QR or geo-location attendance capture; email or SMS notifications; automatic conversion of an approved leave into a "leave" attendance mark; payment or examination modules; stored procedures and triggers; a mobile application.
Assumptions	A single MySQL 8.0 instance is available locally; the administrator creates faculty, students, courses and sections; faculty teach the sections assigned to them; the attendance threshold is 75% unless changed in the settings table or the environment file; demonstration data comes from the supplied seed script.

1.4 Proposed Solution Strategy

The proposed solution models the college's attendance and leave process in a relational database and exposes it through a web application. Table 2 summarises the users, modules, workflow, rules and outputs. The entity-relationship (ER) model [1] was converted to relations, normalized, and implemented in MySQL before the application was built on top of it.

Table 2. Proposed Solution Strategy

Primary Users / Roles	Administrator (full management and all reports); Faculty (own sections, sessions, attendance, makeup sessions, leave review, reports); Student (own attendance, leave applications, makeup schedule, alerts).
Major Application Modules	Authentication and profile; dashboard; students; faculty and departments; courses, sections and enrolments; attendance; leave applications; makeup sessions; reports.
Proposed Workflow	Administrator creates sections and enrols students → faculty creates a class session and marks each student present, absent or leave → the system recomputes attendance and raises or resolves a shortage alert → a student submits a leave request → a reviewer approves or rejects it and the decision is stored → reports and dashboards summarise the data.
Important Business Rules	One attendance mark per student per session; a student can be enrolled in a section once; leave end date cannot precede start date; a leave request can be decided only while pending; attendance % = present ÷ (present + absent + leave) × 100; an alert is raised only after at least two counted classes and when the percentage is below the threshold.
Expected Reports / Outputs	Student-wise and course-wise attendance, shortage roster, leave history, pending leave queue, faculty allocation, makeup participation and monthly trend, each viewable on screen and downloadable as CSV.

1.5 Functional Requirements

The functional requirements in Table 3 are limited to features present in the implemented application.

Table 3. Functional Requirements

Req. ID	Functional Requirement	Remarks / Priority
FR-01	Authenticate users by username or e-mail and password, reject deactivated accounts, and restrict each page to the permitted roles.	High
FR-02	Allow the administrator to add and edit students, faculty, departments, courses and sections, and to activate or deactivate students.	High
FR-03	Enrol students into sections, enforce section capacity and a single enrolment per student per section, and allow an enrolment to be marked dropped.	High
FR-04	Allow a faculty member (or the administrator) to create class sessions for an assigned section.	High
FR-05	Record attendance for every enrolled student as present, absent or leave, and allow later correction of the same record.	High
FR-06	Compute attendance percentage per student per course and raise or resolve shortage alerts against the configured threshold.	High
FR-07	Allow a student to submit a leave request with type, dates, reason and an optional document (PNG, JPG, PDF, DOC, DOCX).	High
FR-08	Allow faculty or the administrator to approve or reject a pending leave request and store the decision, reviewer and notes.	High
FR-09	Schedule makeup sessions and record makeup attendance for the enrolled students of a section.	Medium
FR-10	Provide eight SQL reports with department/course filters where applicable and CSV export.	Medium
FR-11	Show role-specific dashboards with summary counts and Chart.js charts.	Medium
FR-12	Record significant actions (login, logout, session creation, attendance, leave and password events) in an audit log.	Medium

1.6 Entity-Relationship Diagram (ERD)

Figure 1 (on the next page, in landscape orientation for legibility) shows the ER diagram of the implemented database, generated directly from the foreign keys declared in schema.sql. Each box is an entity (table) showing its primary key (PK), foreign keys (FK) and a few identifying attributes; the complete attribute list is in Table 8. A line labelled 1 at one end and N at the other is a one-to-many relationship. The many-to-many relationships between students and sections, sessions and students, and makeup sessions and students are resolved through the enrollments, attendance and makeup_attendance tables.

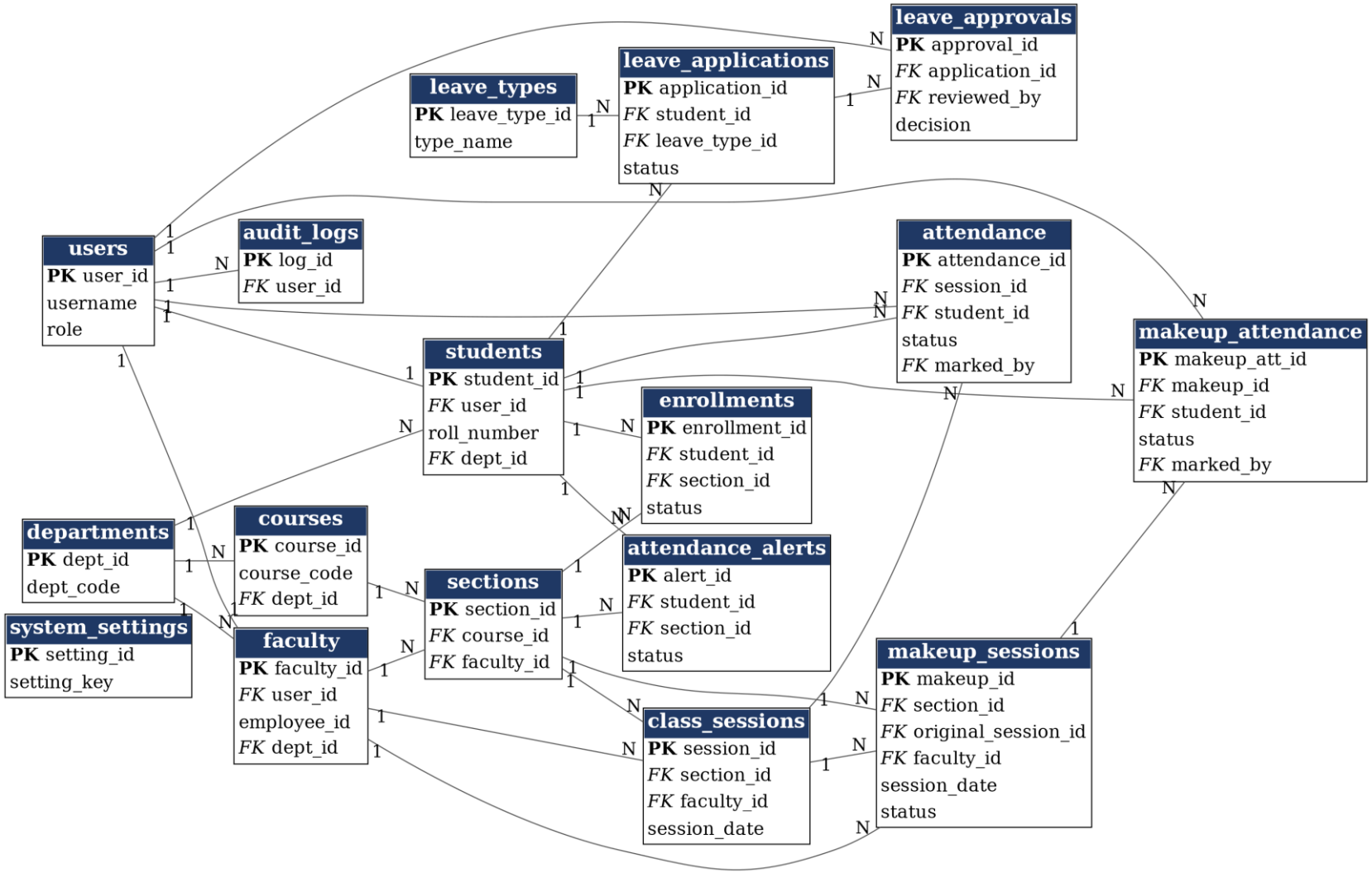


Figure 1. Entity-Relationship Diagram of the Proposed System

1.7 Entity and Relationship Summary

Table 4 lists the principal entities and relationships and their cardinalities as implemented in the schema.

Table 4. Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
users	user_id (PK), username, email, password_hash, role	Single login identity with role admin, faculty or student.
users – students / faculty	1 : 1 (UNIQUE user_id in each profile table)	A student or faculty profile belongs to exactly one user account.
departments	dept_id (PK), dept_code, dept_name	Owns students, faculty and courses (1 : N each).
courses – sections	1 : N (sections.course_id)	A course can be offered as several sections per term and year.
faculty – sections	1 : N (sections.faculty_id)	Each section is assigned to one faculty member.
students – sections (enrollments)	M : N resolved by enrollments(student_id, section_id)	A student can take many sections; a section has many students; status enrolled / dropped / completed.
sections – class_sessions	1 : N	Each conducted lecture, lab or extra class belongs to one section.
class_sessions – students (attendance)	M : N resolved by attendance(session_id, student_id)	One mark (present / absent / leave) per student per session; marked_by references users.
students – leave_applications	1 : N	A student may submit many leave requests.
leave_types – leave_applications	1 : N	Each request is of one leave type (e.g. Medical).
leave_applications – leave_approvals	1 : N by schema (one row per decision in the application)	Stores reviewer, decision and notes; reviewed_by references users.
sections – makeup_sessions	1 : N; optional link to an original class session	Supplementary classes; original_session_id may be NULL.
makeup_sessions – students (makeup_attendance)	M : N resolved by makeup_attendance(makeup_id, student_id)	Records present / absent for makeup classes.
attendance_alerts	N : 1 to students and to sections	Stores the percentage snapshot when a student falls below the threshold.
audit_logs; system_settings	audit_logs N : 1 to users (SET NULL); system_settings standalone	Action trail; key-value policy settings such as min_attendance_threshold.

CHAPTER 2
LOGICAL DATABASE DESIGN

This chapter documents how the ER model of Chapter 1 was converted into relations, how keys and constraints enforce the business rules, and how the design was verified by implementing it in MySQL. All details are taken from schema.sql and seed.sql in the project repository.

2.1 Relational Schema

The ER model yields seventeen relations and two views (Table 5). Primary keys are single-column AUTO_INCREMENT surrogate keys; relationships are implemented as foreign keys. The views v_student_course_attendance and v_section_summary are reporting objects and are not base tables.

Table 5. Relational Schema

S.No.	Table Name	Purpose
1	system_settings	Key-value policy settings (e.g. min_attendance_threshold, academic_year). PK setting_id.
2	departments	College departments. PK dept_id; UNIQUE dept_code.
3	users	Login identities and roles. PK user_id; UNIQUE username, email.
4	students	Student profile. PK student_id; FK user_id → users, dept_id → departments.
5	faculty	Faculty profile. PK faculty_id; FK user_id → users, dept_id → departments.
6	courses	Course master. PK course_id; FK dept_id → departments.
7	sections	Course offering per term and year with faculty, room and capacity. PK section_id; FK course_id, faculty_id.
8	enrollments	Student–section registration (resolves M:N). PK enrollment_id; FK student_id, section_id.
9	class_sessions	Conducted classes. PK session_id; FK section_id, faculty_id.
10	attendance	Per-student mark for a session (resolves M:N). PK attendance_id; FK session_id, student_id, marked_by → users.
11	leave_types	Leave categories with per-term day limits. PK leave_type_id; UNIQUE type_name.
12	leave_applications	Student leave requests. PK application_id; FK student_id, leave_type_id.
13	leave_approvals	Decision audit for a leave request. PK approval_id; FK application_id, reviewed_by → users.
14	makeup_sessions	Supplementary classes. PK makeup_id; FK section_id, original_session_id (nullable), faculty_id.
15	makeup_attendance	Per-student makeup mark (resolves M:N). PK makeup_att_id; FK makeup_id, student_id, marked_by.
16	attendance_alerts	Shortage alert snapshots. PK alert_id; FK student_id, section_id.
17	audit_logs	Action trail. PK log_id; FK user_id → users (nullable).

Views: v_student_course_attendance (per enrolment: counts of present, absent and leave marks and the attendance percentage) and v_section_summary (per section: enrolled students and sessions conducted).

2.2 Keys and Integrity Constraints

Table 6 lists the keys and integrity constraints declared in schema.sql, with the rule each one enforces. Every mandatory attribute is declared NOT NULL; these are shown per column in Table 8. Referential actions follow a consistent policy: ON DELETE CASCADE for dependent detail rows (for example attendance rows of a deleted session), ON DELETE RESTRICT where history must not be orphaned (for example a faculty member who is assigned to a section), and ON DELETE SET NULL for optional links (audit_logs.user_id and makeup_sessions.original_session_id). The application itself never issues DELETE statements; removal is represented by is_active = FALSE or status = 'dropped'.

Table 6. Keys and Integrity Constraints

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
system_settings	Primary Key	setting_id	Uniquely identifies each row.
	Unique	setting_key	Each configuration key appears once.
	Default	updated_at=CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Supplies a value when none is given.
departments	Primary Key	dept_id	Uniquely identifies each row.
	Unique	dept_code	No two departments share a code.
	Default	created_at=CURRENT_TIMESTAMP	Supplies a value when none is given.
users	Primary Key	user_id	Uniquely identifies each row.
	Unique	username	Unique login name.
	Unique	email	Unique login e-mail.
	Check (ENUM)	role {admin, faculty, student}	Restricts role to the listed values.
	Default	is_active=TRUE; created_at=CURRENT_TIMESTAMP; updated_at=CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Supplies a value when none is given.
students	Primary Key	student_id	Uniquely identifies each row.
	Foreign Key	user_id → users(user_id) ON DELETE CASCADE	user_id must match an existing users row.
	Foreign Key	dept_id → departments(dept_id) ON DELETE RESTRICT	dept_id must match an existing departments row.
	Unique	user_id	One student profile per user account (1 : 1).
	Unique	roll_number	Roll numbers are unique.
	Unique	email	Student e-mail is unique.

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
	Default	current_semester=1; is_active=TRUE; created_at=CURRENT_TIMESTAMP	Supplies a value when none is given.
faculty	Primary Key	faculty_id	Uniquely identifies each row.
	Foreign Key	user_id → users(user_id) ON DELETE CASCADE	user_id must match an existing users row.
	Foreign Key	dept_id → departments(dept_id) ON DELETE RESTRICT	dept_id must match an existing departments row.
	Unique	user_id	One faculty profile per user account (1 : 1).
	Unique	employee_id	Employee IDs are unique.
	Unique	email	Faculty e-mail is unique.
	Default	designation=Assistant Professor; is_active=TRUE; created_at=CURRENT_TIMESTAMP	Supplies a value when none is given.
courses	Primary Key	course_id	Uniquely identifies each row.
	Foreign Key	dept_id → departments(dept_id) ON DELETE RESTRICT	dept_id must match an existing departments row.
	Unique	course_code	Course codes are unique.
	Default	credits=3; is_active=TRUE; created_at=CURRENT_TIMESTAMP	Supplies a value when none is given.
sections	Primary Key	section_id	Uniquely identifies each row.
	Foreign Key	course_id → courses(course_id) ON DELETE RESTRICT	course_id must match an existing courses row.
	Foreign Key	faculty_id → faculty(faculty_id) ON DELETE RESTRICT	faculty_id must match an existing faculty row.
	Unique	course_id, section_name, academic_term, academic_year	A section name cannot repeat for a course in the same term and year.
	Default	max_capacity=60; is_active=TRUE; created_at=CURRENT_TIMESTAMP	Supplies a value when none is given.
enrollments	Primary Key	enrollment_id	Uniquely identifies each row.
	Foreign Key	student_id → students(student_id) ON DELETE CASCADE	student_id must match an existing students row.
	Foreign Key	section_id → sections(section_id) ON DELETE CASCADE	section_id must match an existing sections row.
	Unique	student_id, section_id	A student is enrolled in a section at most once.
	Check (ENUM)	status {enrolled, dropped, completed}	Restricts status to the listed values.
	Default	enrolled_at=CURRENT_TIMESTAMP; status=enrolled	Supplies a value when none is given.
class_sessions	Primary Key	session_id	Uniquely identifies each row.
	Foreign Key	section_id → sections(section_id) ON DELETE CASCADE	section_id must match an existing sections row.
	Foreign Key	faculty_id → faculty(faculty_id) ON DELETE RESTRICT	faculty_id must match an existing faculty row.
	Check (ENUM)	session_type {regular, extra, lab}	Restricts session_type to the listed values.
	Default	session_type=regular; created_at=CURRENT_TIMESTAMP	Supplies a value when none is given.

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
attendance	Primary Key	attendance_id	Uniquely identifies each row.
	Foreign Key	session_id → class_sessions(session_id) ON DELETE CASCADE	session_id must match an existing class_sessions row.
	Foreign Key	student_id → students(student_id) ON DELETE CASCADE	student_id must match an existing students row.
	Foreign Key	marked_by → users(user_id) ON DELETE RESTRICT	marked_by must match an existing users row.
	Unique	session_id, student_id	At most one attendance mark per student per session (prevents duplicates).
	Check (ENUM)	status {present, absent, leave}	Restricts status to the listed values.
	Default	marked_at=CURRENT_TIMESTAMP	Supplies a value when none is given.
leave_types	Primary Key	leave_type_id	Uniquely identifies each row.
	Unique	type_name	Leave type names are unique.
	Default	max_days_per_term=10; is_active=TRUE	Supplies a value when none is given.
leave_applications	Primary Key	application_id	Uniquely identifies each row.
	Foreign Key	student_id → students(student_id) ON DELETE CASCADE	student_id must match an existing students row.
	Foreign Key	leave_type_id → leave_types(leave_type_id) ON DELETE RESTRICT	leave_type_id must match an existing leave_types row.
	Check	end_date >= start_date	A leave cannot end before it starts.
	Check (ENUM)	status {pending, approved, rejected}	Restricts status to the listed values.
	Default	total_days=1; status=pending; applied_at=CURRENT_TIMESTAMP; updated_at=CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Supplies a value when none is given.
leave_approvals	Primary Key	approval_id	Uniquely identifies each row.
	Foreign Key	application_id → leave_applications(application_id) ON DELETE CASCADE	application_id must match an existing leave_applications row.
	Foreign Key	reviewed_by → users(user_id) ON DELETE RESTRICT	reviewed_by must match an existing users row.
	Check (ENUM)	decision {approved, rejected}	Restricts decision to the listed values.
	Default	reviewed_at=CURRENT_TIMESTAMP	Supplies a value when none is given.
makeup_sessions	Primary Key	makeup_id	Uniquely identifies each row.
	Foreign Key	section_id → sections(section_id) ON DELETE CASCADE	section_id must match an existing sections row.
	Foreign Key	original_session_id → class_sessions(session_id) ON DELETE SET NULL	original_session_id must match an existing class_sessions row.
	Foreign Key	faculty_id → faculty(faculty_id) ON DELETE RESTRICT	faculty_id must match an existing faculty row.
	Check (ENUM)	status {scheduled, completed, cancelled}	Restricts status to the listed values.

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
	Default	status=scheduled; created_at=CURRENT_TIMESTAMP	Supplies a value when none is given.
makeup_attendance	Primary Key	makeup_att_id	Uniquely identifies each row.
	Foreign Key	makeup_id → makeup_sessions(makeup_id) ON DELETE CASCADE	makeup_id must match an existing makeup_sessions row.
	Foreign Key	student_id → students(student_id) ON DELETE CASCADE	student_id must match an existing students row.
	Foreign Key	marked_by → users(user_id) ON DELETE RESTRICT	marked_by must match an existing users row.
	Unique	makeup_id, student_id	One makeup mark per student per makeup session.
	Check (ENUM)	status {present, absent}	Restricts status to the listed values.
	Default	status=present; marked_at=CURRENT_TIMESTAMP	Supplies a value when none is given.
attendance_alerts	Primary Key	alert_id	Uniquely identifies each row.
	Foreign Key	student_id → students(student_id) ON DELETE CASCADE	student_id must match an existing students row.
	Foreign Key	section_id → sections(section_id) ON DELETE CASCADE	section_id must match an existing sections row.
	Check (ENUM)	status {active, resolved, acknowledged}	Restricts status to the listed values.

	Default	threshold_percentage=75.00;
		status=active;
		created_at=CURRENT_TIMESTAMP;
		updated_at=CURRENT_TIMESTAMP
		ON UPDATE CURRENT_TIMESTAMP
			Supplies a value when none is given.

audit_logs	Primary Key	log_id	Uniquely identifies each row.
	Foreign Key	user_id → users(user_id) ON DELETE SET NULL	user_id must match an existing users row.
	Default	created_at=CURRENT_TIMESTAMP	Supplies a value when none is given.

Indexes are declared on frequently filtered columns (for example attendance.student_id and attendance.status, class_sessions.session_date, leave_applications.status) to support the joins and filters used by the reports.

2.3 Functional Dependencies

The functional dependencies below follow from the declared primary keys and unique constraints of the implemented tables. Each determinant is a key (primary or candidate) unless noted in Section 2.4.

- departments: dept_id → dept_code, dept_name; dept_code → dept_id
- users: user_id → username, email, password_hash, role, is_active; username → user_id; email → user_id

- students: student_id → user_id, roll_number, full_name, email, dept_id, program, current_semester, phone, is_active; roll_number → student_id; user_id → student_id
- faculty: faculty_id → user_id, employee_id, full_name, email, dept_id, designation, phone; employee_id → faculty_id
- courses: course_id → course_code, course_title, credits, dept_id; course_code → course_id
- sections: section_id → course_id, faculty_id, section_name, academic_term, academic_year, classroom, max_capacity; (course_id, section_name, academic_term, academic_year) → section_id
- enrollments: enrollment_id → student_id, section_id, enrolled_at, status; (student_id, section_id) → enrollment_id
- class_sessions: session_id → section_id, faculty_id, session_date, start_time, end_time, topic, session_type
- attendance: attendance_id → session_id, student_id, status, remarks, marked_at, marked_by; (session_id, student_id) → attendance_id
- leave_types: leave_type_id → type_name, max_days_per_term; type_name → leave_type_id
- leave_applications: application_id → student_id, leave_type_id, start_date, end_date, total_days, reason, document_url, status; (start_date, end_date) → total_days
- leave_approvals: approval_id → application_id, reviewed_by, decision, decision_notes, reviewed_at
- makeup_sessions: makeup_id → section_id, original_session_id, faculty_id, session_date, start_time, end_time, classroom, topic, status
- makeup_attendance: makeup_att_id → makeup_id, student_id, status, marked_by; (makeup_id, student_id) → makeup_att_id
- attendance_alerts: alert_id → student_id, section_id, current_percentage, threshold_percentage, classes_attended, classes_missed, status
- audit_logs: log_id → user_id, action, entity_type, entity_id, details, ip_address, created_at
- system_settings: setting_id → setting_key, setting_value, description; setting_key → setting_id

2.4 Normalization up to Third Normal Form (3NF)

Normalization removes redundancy and update anomalies by ensuring that every non-key attribute depends on the key, the whole key, and nothing but the key [1], [2]. All seventeen relations hold atomic values (1NF). Because every relation has a single-column surrogate primary key, no partial dependency on part of a key is possible, so 2NF holds. In every relation

the non-key attributes depend directly on the key, so 3NF holds except for the three deliberate exceptions documented below. Table 7 gives the status of each relation.

Table 7. Normalization Status up to 3NF

Relation Name	1NF	2NF	3NF	BCNF	Remarks / Decomposition
system_settings	Yes	Yes	Yes	Yes	No transitive dependency.
departments	Yes	Yes	Yes	Yes	Both dept_id and dept_code are candidate keys.
users	Yes	Yes	Yes	Yes	Credentials and role depend only on user_id.
students	Yes	Yes	Yes	Yes*	*email is duplicated from users.email; both are unique and kept in step by the edit routes.
faculty	Yes	Yes	Yes	Yes*	*Same e-mail duplication as students.
courses	Yes	Yes	Yes	Yes	Department attributes stored once in departments.
sections	Yes	Yes	Yes	Yes	Course and faculty details are referenced, not repeated.
enrollments	Yes	Yes	Yes	Yes	Junction table; (student_id, section_id) is a candidate key.
class_sessions	Yes	Yes	Yes	See note	faculty_id may differ from sections.faculty_id (substitute teacher), so it is not a transitive dependency by design.
attendance	Yes	Yes	Yes	Yes	Junction table with a unique (session_id, student_id).
leave_types	Yes	Yes	Yes	Yes	Limits stored once per type.
leave_applications	Yes	Yes	Exception	No	total_days is derivable from start_date and end_date (derived attribute kept for convenient reporting); to remove it, compute it in queries.
leave_approvals	Yes	Yes	Yes	Yes	Decision data depends on approval_id.
makeup_sessions	Yes	Yes	Yes	Yes	classroom is stored per makeup session because it can differ from the section room.
makeup_attendance	Yes	Yes	Yes	Yes	Junction table with a unique (makeup_id, student_id).
attendance_alerts	Yes	Yes	Exception	No	Percentage and counts are derived from attendance; stored deliberately as a point-in-time alert snapshot.
audit_logs	Yes	Yes	Yes	Yes	Append-only log; entity_type / entity_id is a soft reference.

The exceptions are intentional trade-offs rather than oversights. Removing leave_applications.total_days and attendance_alerts snapshot columns would give strict 3NF at the cost of recomputation in each query; keeping them risks inconsistency if dates or attendance are edited without refreshing the derived values. In the current application, total_days is computed once when the application is inserted, and alerts are refreshed after each attendance save.

2.5 Data Dictionary

Table 8 is the data dictionary of all 133 columns in the 17 tables. It is generated from the declarations in schema.sql; “Key / Constraint” lists keys, uniqueness, NOT NULL, ENUM domains and defaults.

Table 8. Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
system_settings	setting_id	INT	PK; AUTO_INCREMENT	Surrogate primary key.
	setting_key	VARCHAR(50)	UNIQUE; NOT NULL	Configuration key.
	setting_value	VARCHAR(255)	NOT NULL	Configuration value (text).
	description	VARCHAR(255)		Free-text description.
	updated_at	TIME	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Last modification timestamp (auto-updated).
departments	dept_id	INT	PK; AUTO_INCREMENT	Surrogate primary key.
	dept_code	VARCHAR(20)	UNIQUE; NOT NULL	Short code, e.g. CSE.
	dept_name	VARCHAR(150)	NOT NULL	Full department name.
	created_at	TIME	DEFAULT CURRENT_TIMESTAMP	Row creation timestamp.
users	user_id	INT	PK; AUTO_INCREMENT	Surrogate primary key.
	username	VARCHAR(50)	UNIQUE; NOT NULL	Login name.
	email	VARCHAR(120)	UNIQUE; NOT NULL	E-mail address.
	password_hash	VARCHAR(255)	NOT NULL	Scrypt hash produced by Werkzeug; plain password is never stored.
	role	ENUM	NOT NULL; {admin, faculty, student}	Role used for access control.
	is_active	BOOLEAN	DEFAULT TRUE	Active flag used instead of deleting the row.
	created_at	TIME	DEFAULT CURRENT_TIMESTAMP	Row creation timestamp.
	updated_at	TIME	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Last modification timestamp (auto-updated).
students	student_id	INT	PK; AUTO_INCREMENT	Surrogate primary key.
	user_id	INT	FK → users(user_id); UNIQUE; NOT NULL	References users.
	roll_number	VARCHAR(50)	UNIQUE; NOT NULL	Institutional roll number.
	full_name	VARCHAR(120)	NOT NULL	Full name.
	email	VARCHAR(120)	UNIQUE; NOT NULL	E-mail address.
	dept_id	INT	FK → departments(dept_id); NOT NULL	References departments.
	program	VARCHAR(100)	NOT NULL	Degree programme name.
	current_semester	INT	NOT NULL; DEFAULT 1	Current semester number.
	phone	VARCHAR(25)		Contact number (optional).
	is_active	BOOLEAN	DEFAULT TRUE	Active flag used instead of deleting the row.
	created_at	TIME	DEFAULT CURRENT_TIMESTAMP	Row creation timestamp.
faculty	faculty_id	INT	PK; AUTO_INCREMENT	Surrogate primary key.

Table	Attribute	Data Type	Key / Constraint	Description
	user_id	INT	FK → users(user_id); UNIQUE; NOT NULL	References users.
	employee_id	VARCHAR(50)	UNIQUE; NOT NULL	Institutional employee ID.
	full_name	VARCHAR(120)	NOT NULL	Full name.
	email	VARCHAR(120)	UNIQUE; NOT NULL	E-mail address.
	dept_id	INT	FK → departments(dept_id); NOT NULL	References departments.
	designation	VARCHAR(100)	NOT NULL; DEFAULT Assistant Professor	Academic designation.
	phone	VARCHAR(25)		Contact number (optional).
	is_active	BOOLEAN	DEFAULT TRUE	Active flag used instead of deleting the row.
	created_at	TIME	DEFAULT CURRENT_TIMESTAMP	Row creation timestamp.
courses	course_id	INT	PK; AUTO_INCREMENT	Surrogate primary key.
	course_code	VARCHAR(30)	UNIQUE; NOT NULL	Course code, e.g. CS301.
	course_title	VARCHAR(150)	NOT NULL	Course title.
	credits	INT	NOT NULL; DEFAULT 3	Credit value.
	dept_id	INT	FK → departments(dept_id); NOT NULL	References departments.
	is_active	BOOLEAN	DEFAULT TRUE	Active flag used instead of deleting the row.
	created_at	TIME	DEFAULT CURRENT_TIMESTAMP	Row creation timestamp.
sections	section_id	INT	PK; AUTO_INCREMENT	Surrogate primary key.
	course_id	INT	FK → courses(course_id); NOT NULL	References courses.
	faculty_id	INT	FK → faculty(faculty_id); NOT NULL	References faculty.
	section_name	VARCHAR(50)	NOT NULL	Section label, e.g. CS301-A.
	academic_term	VARCHAR(30)	NOT NULL	Term, e.g. Fall.
	academic_year	VARCHAR(20)	NOT NULL	Academic year, e.g. 2026-2027.
	classroom	VARCHAR(50)	NOT NULL	Room name.
	max_capacity	INT	NOT NULL; DEFAULT 60	Maximum students allowed.
	is_active	BOOLEAN	DEFAULT TRUE	Active flag used instead of deleting the row.
	created_at	TIME	DEFAULT CURRENT_TIMESTAMP	Row creation timestamp.
enrollments	enrollment_id	INT	PK; AUTO_INCREMENT	Surrogate primary key.
	student_id	INT	FK → students(student_id); NOT NULL	References students.
	section_id	INT	FK → sections(section_id); NOT NULL	References sections.
	enrolled_at	TIME	DEFAULT CURRENT_TIMESTAMP	Enrolment time.

Table	Attribute	Data Type	Key / Constraint	Description
	status	ENUM	NOT NULL; {enrolled, dropped, completed}; DEFAULT enrolled	Enrolment state.
class_sessions	session_id	INT	PK; AUTO_INCREMENT	Surrogate primary key.
	section_id	INT	FK → sections(section_id); NOT NULL	References sections.
	faculty_id	INT	FK → faculty(faculty_id); NOT NULL	References faculty.
	session_date	DATE	NOT NULL	Date of the class.
	start_time	TIME	NOT NULL	Start time.
	end_time	TIME	NOT NULL	End time.
	topic	VARCHAR(255)		Topic covered.
	session_type	ENUM	NOT NULL; {regular, extra, lab}; DEFAULT regular	Kind of session.
	created_at	TIME	DEFAULT CURRENT_TIMESTAMP	Row creation timestamp.
attendance	attendance_id	INT	PK; AUTO_INCREMENT	Surrogate primary key.
	session_id	INT	FK → class_sessions(session_id); NOT NULL	References class_sessions.
	student_id	INT	FK → students(student_id); NOT NULL	References students.
	status	ENUM	NOT NULL; {present, absent, leave}	Attendance state of the student.
	remarks	VARCHAR(255)		Optional remark.
	marked_at	TIME	DEFAULT CURRENT_TIMESTAMP	Time the mark was recorded.
	marked_by	INT	FK → users(user_id); NOT NULL	User who recorded the mark.
leave_types	leave_type_id	INT	PK; AUTO_INCREMENT	Surrogate primary key.
	type_name	VARCHAR(50)	UNIQUE; NOT NULL	Leave category name.
	max_days_per_term	INT	NOT NULL; DEFAULT 10	Days allowed per term for this type.
	description	VARCHAR(255)		Free-text description.
	is_active	BOOLEAN	DEFAULT TRUE	Active flag used instead of deleting the row.
leave_applications	application_id	INT	PK; AUTO_INCREMENT	Surrogate primary key.
	student_id	INT	FK → students(student_id); NOT NULL	References students.
	leave_type_id	INT	FK → leave_types(leave_type_id); NOT NULL	References leave_types.
	start_date	DATE	NOT NULL	First day of leave.
	end_date	DATE	NOT NULL	Last day of leave.
	total_days	INT	NOT NULL; DEFAULT 1	Number of leave days (derived, inclusive).
	reason	TEXT	NOT NULL	Reason given by the student.
	document_url	VARCHAR(255)		Stored file name of the uploaded document.
	status	ENUM	NOT NULL; {pending, approved, rejected}; DEFAULT pending	Workflow state of the request.

Table	Attribute	Data Type	Key / Constraint	Description
	applied_at	TIME	DEFAULT CURRENT_TIMESTAMP	Submission time.
	updated_at	TIME	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Last modification timestamp (auto-updated).
leave_approvals	approval_id	INT	PK; AUTO_INCREMENT	Surrogate primary key.
	application_id	INT	FK → leave_applications(application_id); NOT NULL	References leave_applications.
	reviewed_by	INT	FK → users(user_id); NOT NULL	User who reviewed the request.
	decision	ENUM	NOT NULL; {approved, rejected}	Reviewer decision.
	decision_notes	TEXT		Reviewer's notes.
	reviewed_at	TIME	DEFAULT CURRENT_TIMESTAMP	Decision time.
makeup_sessions	makeup_id	INT	PK; AUTO_INCREMENT	Surrogate primary key.
	section_id	INT	FK → sections(section_id); NOT NULL	References sections.
	original_session_id	INT	FK → class_sessions(session_id)	Class session being made up (optional).
	faculty_id	INT	FK → faculty(faculty_id); NOT NULL	References faculty.
	session_date	DATE	NOT NULL	Date of the class.
	start_time	TIME	NOT NULL	Start time.
	end_time	TIME	NOT NULL	End time.
	classroom	VARCHAR(50)	NOT NULL	Room name.
	topic	VARCHAR(255)	NOT NULL	Topic covered.
	status	ENUM	NOT NULL; {scheduled, completed, cancelled}; DEFAULT scheduled	State of the makeup session.
	created_at	TIME	DEFAULT CURRENT_TIMESTAMP	Row creation timestamp.
makeup_attendance	makeup_att_id	INT	PK; AUTO_INCREMENT	Surrogate primary key.
	makeup_id	INT	FK → makeup_sessions(makeup_id); NOT NULL	References makeup_sessions.
	student_id	INT	FK → students(student_id); NOT NULL	References students.
	status	ENUM	NOT NULL; {present, absent}; DEFAULT present	Makeup attendance state.
	marked_at	TIME	DEFAULT CURRENT_TIMESTAMP	Time the mark was recorded.
	marked_by	INT	FK → users(user_id); NOT NULL	User who recorded the mark.
attendance_alerts	alert_id	INT	PK; AUTO_INCREMENT	Surrogate primary key.
	student_id	INT	FK → students(student_id); NOT NULL	References students.
	section_id	INT	FK → sections(section_id); NOT NULL	References sections.

Table	Attribute	Data Type	Key / Constraint	Description
	current_percentage	DECIMAL(5,2)	NOT NULL	Attendance % when the alert was computed.
	threshold_percentage	DECIMAL(5,2)	NOT NULL; DEFAULT 75.00	Threshold in force.
	classes_attended	INT	NOT NULL	Present count.
	classes_missed	INT	NOT NULL	Absent count.
	status	ENUM	NOT NULL; {active, resolved, acknowledged}; DEFAULT active	Alert state.
	created_at	TIME	DEFAULT CURRENT_TIMESTAMP	Row creation timestamp.
	updated_at	TIME	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Last modification timestamp (auto-updated).
audit_logs	log_id	INT	PK; AUTO_INCREMENT	Surrogate primary key.
	user_id	INT	FK → users(user_id)	References users.
	action	VARCHAR(100)	NOT NULL	Action code, e.g. USER_LOGIN.
	entity_type	VARCHAR(50)	NOT NULL	Table or object affected.
	entity_id	INT		Affected row ID (soft reference).
	details	TEXT		Free-text details.
	ip_address	VARCHAR(45)		Client IP address.
	created_at	TIME	DEFAULT CURRENT_TIMESTAMP	Row creation timestamp.

2.6 Implementation of Logical Design: DDL and Sample Data

The design was implemented in MySQL 8.0 with the InnoDB engine, which supports foreign keys and transactions [3]. Table 9 summarises the environment, and the complete script is described in Appendix A.

Table 9. Database Implementation Environment

DBMS	MySQL
DBMS Version	8.0 (schema.sql is written for MySQL 8.0+; CHECK constraints require 8.0.16 or later)
Database Name / Character Set	attendease_db / utf8mb4 (collation utf8mb4_unicode_ci)
Storage Engine	InnoDB
Number of Tables	17 base tables and 2 views
Sample Data	seed.sql: 4 departments, 10 users (1 admin, 3 faculty, 6 students), 3 faculty, 6 students, 4 courses, 4 sections, 11 enrolments, 12 class sessions, 32 attendance marks, 4 leave types, 3 leave applications, 2 approvals, 2 makeup sessions, 2 makeup attendance rows, 1 alert, 5 audit rows, 4 settings.

Representative DDL is shown below: the students table (profile linked 1 : 1 to users and N : 1 to departments), the attendance table (junction table with the duplicate-prevention constraint), and the leave_applications table (CHECK constraint on dates).

```
CREATE TABLE students (
    student_id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL UNIQUE,
    roll_number VARCHAR(50) NOT NULL UNIQUE,
    full_name VARCHAR(120) NOT NULL,
    email VARCHAR(120) NOT NULL UNIQUE,
    dept_id INT NOT NULL,
    program VARCHAR(100) NOT NULL,
    current_semester INT NOT NULL DEFAULT 1,
    phone VARCHAR(25) NULL,
    is_active BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_students_user FOREIGN KEY (user_id)
        REFERENCES users(user_id) ON DELETE CASCADE,
    CONSTRAINT fk_students_dept FOREIGN KEY (dept_id)
        REFERENCES departments(dept_id) ON DELETE RESTRICT,
    INDEX idx_student_dept (dept_id),
    INDEX idx_student_roll (roll_number)
) ENGINE=InnoDB;
```

```
CREATE TABLE attendance (
    attendance_id INT AUTO_INCREMENT PRIMARY KEY,
    session_id INT NOT NULL,
    student_id INT NOT NULL,
    status ENUM('present', 'absent', 'leave') NOT NULL,
    remarks VARCHAR(255) NULL,
    marked_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    marked_by INT NOT NULL,
    CONSTRAINT fk_attendance_session FOREIGN KEY (session_id)
        REFERENCES class_sessions(session_id) ON DELETE CASCADE,
    CONSTRAINT fk_attendance_student FOREIGN KEY (student_id)
        REFERENCES students(student_id) ON DELETE CASCADE,
    CONSTRAINT fk_attendance_marker FOREIGN KEY (marked_by)
        REFERENCES users(user_id) ON DELETE RESTRICT,
    CONSTRAINT uq_session_student UNIQUE (session_id, student_id),
    INDEX idx_att_student (student_id),
    INDEX idx_att_status (status)
) ENGINE=InnoDB;
```

```
CREATE TABLE leave_applications (
    application_id INT AUTO_INCREMENT PRIMARY KEY,
    student_id INT NOT NULL,
    leave_type_id INT NOT NULL,
    start_date DATE NOT NULL,
    end_date DATE NOT NULL,
    total_days INT NOT NULL DEFAULT 1,
    reason TEXT NOT NULL,
    document_url VARCHAR(255) NULL,
    status ENUM('pending', 'approved', 'rejected') NOT NULL DEFAULT 'pending',
    applied_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    CONSTRAINT fk_leave_student FOREIGN KEY (student_id)
        REFERENCES students(student_id) ON DELETE CASCADE,
    CONSTRAINT fk_leave_type FOREIGN KEY (leave_type_id)
        REFERENCES leave_types(leave_type_id) ON DELETE RESTRICT,
```

```
    CONSTRAINT chk_leave_dates CHECK (end_date >= start_date),
    INDEX idx_leave_student (student_id),
    INDEX idx_leave_status (status)
) ENGINE=InnoDB;
```

Sample data in seed.sql demonstrates each attendance outcome. For section CS301-A (eight sessions), Alex Rivera is present in all eight (100%), Priya Patel has seven presents and one leave (87.5%), Elena Rostova has six presents and two absences (75.0%, exactly at the threshold), and Marcus Vance has four presents and four absences (50.0%) with an active shortage alert. A representative extract is shown below.

```
INSERT INTO attendance (session_id, student_id, status, remarks, marked_by)
VALUES
(3, 1, 'present', 'On time', 2),
(3, 2, 'leave',   'Approved Medical Leave #1', 2),
(3, 3, 'absent',  'Unexcused', 2),
(3, 4, 'present', 'On time', 2);

INSERT INTO leave_applications (application_id, student_id, leave_type_id,
start_date,
  end_date, total_days, reason, status, applied_at)
VALUES (1, 2, 1, '2026-10-05', '2026-10-05', 1,
  'Severe viral fever and flu symptoms. Prescribed doctor bed rest.',
  'approved', '2026-10-04 18:30:00');
```

The two views encapsulate the attendance calculation. v_student_course_attendance joins enrollments, students, departments, sections, courses and faculty, left-joins class_sessions and attendance, and computes attendance_percentage with ROUND, SUM(CASE …) and NULLIF (to avoid division by zero). Its full definition appears in Appendix A.

CHAPTER 3
APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

This chapter shows how the logical design of Chapter 2 is used by the Flask application. Every screen reads from or writes to the MySQL database; no data is held in static form.

3.1 Application Overview

AttendEase is a server-rendered web application. After login, each user is routed to a dashboard appropriate to the role stored in users.role. Students view their course-wise attendance, shortage alerts, makeup sessions and leave requests, and can submit new leave requests. Faculty create class sessions for their sections, mark attendance, review leave requests, schedule makeup sessions and view reports. Administrators additionally manage departments, students, faculty, courses, sections and enrolments. Each page issues parameterised SQL through a small data-access layer, and the results are rendered with Jinja2 templates and, on dashboards, Chart.js charts.

3.2 Development Environment and Database Connectivity

The backend is written in Python using the Flask framework [4]. It connects to MySQL through the mysql-connector-python driver using a connection pool of five connections with autocommit disabled (database/connection.py). Passwords are hashed with the scrypt-based helpers of Werkzeug [5], and the dashboards use Chart.js version 4.4.1 loaded from a CDN [6]. The source code is hosted in the project repository [7]. Table 10 lists the environment; the pinned versions are those in requirements.txt.

Table 10. Application Development Environment

Programming Language	Python 3.10 or higher
DBMS	MySQL 8.0 (database attendease_db)
IDE / Framework / GUI Toolkit	Flask 3.1.3 with Blueprints and Jinja2 templates; HTML5, CSS3, JavaScript; Chart.js 4.4.1
Database Connector / Driver	mysql-connector-python 26.7.0 (connection pool, parameterised queries); python-dotenv 1.2.4 for configuration; Werkzeug 3.1.9 for password hashing
Operating Environment	Windows with the MySQL80 service (helper scripts init_db.bat and run.bat); server at http://127.0.0.1:5000
Repository Link	https://github.com/atvaholic/DBMS_PBL (as stated in the repository README)

Database access is centralised in four helper functions: query_all and query_one (SELECT), execute (single INSERT, UPDATE or DELETE with commit and rollback on error) and execute_transaction (a list of statements committed together, or rolled back if any fails). Credentials are read from a .env file that is excluded from version control; only a template, .env.example, is supplied.

3.3 Application Modules and Database Mapping

The application is organised into nine Flask blueprints (Table 11). Each blueprint corresponds to a source file in the routes folder.

Table 11. Application Module-to-Database Mapping

Module / Screen	Purpose	Tables Used	Main Operations
Authentication & Profile (auth.py)	Login, logout, profile, change password; RBAC decorators	users, students, faculty, audit_logs	SELECT by username or e-mail; verify hash; UPDATE password_hash; INSERT audit row
Dashboard (dashboard.py)	Role-specific summaries and chart data	students, faculty, courses, sections, class_sessions, attendance, attendance_alerts, leave_applications, makeup_sessions, v_student_course_attendance	COUNT / SUM aggregates; monthly trend; per-role lists
Students (students.py)	Admin list, add, edit, activate / deactivate students	students, users, departments	INSERT users then students; UPDATE profile and e-mail; toggle is_active
Faculty & Departments (faculty.py)	Admin management of faculty and departments	faculty, users, departments	INSERT, UPDATE, SELECT with joins
Courses, Sections, Enrolments (courses.py)	Course and section master; enrolment with capacity check	courses, sections, enrollments, students, faculty, departments	INSERT / UPDATE; duplicate and capacity checks; status enrolled / dropped
Attendance (attendance.py)	Create sessions, mark attendance, student view, alert refresh	class_sessions, attendance, enrollments, attendance_alerts, system_settings, v_student_course_attendance, makeup_attendance	INSERT … ON DUPLICATE KEY UPDATE in a transaction; aggregate percentage; INSERT / UPDATE alerts
Leave Applications (leaves.py)	Submit, list, approve or reject leave; download document	leave_applications, leave_types, leave_approvals, students, enrollments, sections, users, faculty	INSERT application; UPDATE status and INSERT approval in one transaction
Makeup Sessions (makeup.py)	Schedule makeup classes and mark participation	makeup_sessions, makeup_attendance, enrollments, sections	INSERT session; transactional upsert of attendance and status update
Reports (reports.py)	Eight SQL reports and CSV export	v_student_course_attendance, courses, sections, class_sessions, attendance, leave tables, faculty, makeup tables	Joins, GROUP BY, HAVING, derived-table subquery, DATE_FORMAT; CSV streaming

Every base table is referenced by at least one query in the application. The view v_student_course_attendance is queried by the dashboard, student attendance page and Report

1; the view v_section_summary is defined in schema.sql but is not queried by the current routes.

3.4 CRUD Operations

Table 12 maps the CRUD operations to application features. The application performs no physical deletion; "removal" is a controlled status change, which preserves attendance and leave history.

Table 12. CRUD Operations

Operation	Application Feature / Screen	Table(s) Affected	Evidence
Create	Add student / faculty (Students, Faculty screens); create session; submit leave; schedule makeup	users and students (or faculty); class_sessions; leave_applications; makeup_sessions	POST /students/add, /faculty/add, /attendance/sessions/create, /leaves/apply, /makeup/create; Figure 3
Read / Search	Student attendance summary; attendance hub; leave lists with status filter; reports with department / course filters	v_student_course_attendance, attendance, leave_applications, and others	GET /my-attendance, /attendance, /leaves?status=, /reports; Figures 4 and 5
Update	Mark or correct attendance; review leave; edit student, faculty, course, section	attendance, leave_applications, leave_approvals, students, faculty, courses, sections	INSERT … ON DUPLICATE KEY UPDATE; POST /leaves/review/<id>; Figures 2 and 3
Delete / Controlled Removal	Deactivate student; drop enrolment; deactivation blocks login	students.is_active, users.is_active, enrollments.status	POST /students/toggle-status/<id>; POST /sections/<id>/unenroll/<student>; no DELETE statement in routes

3.5 Business Rules, Validation and Error Handling

Validation is applied at three levels: in the browser form, in the Flask route, and in the database through constraints. All SQL uses placeholders (%s) with bound parameters, which protects against SQL injection. Each route is wrapped in exception handling that reports the failure to the user with a flash message. Multi-statement updates use execute_transaction, which commits only if every statement succeeds and otherwise rolls back. Table 13 lists the main rules.

Table 13. Business Rules, Validation and Error Handling

Rule / Validation	How Implemented	Evidence / Message
Login must be valid and active	Look up user by username or e-mail; check_password_hash; reject if is_active is false	"Invalid username/email or password."; "This account has been deactivated…"
Role-based access	login_required and role_required decorators on each route; non-permitted roles redirected to dashboard	"Access denied: You do not have permission for this resource."
Password change rules	All fields required, confirmation must match, minimum 6 characters, current password verified	"New password must be at least 6 characters long."

Rule / Validation	How Implemented	Evidence / Message
Leave dates valid	Route compares end and start dates; database CHECK (end_date >= start_date) as second barrier	“Validation Error: End date cannot be earlier than start date.”
Leave document safety	Extension whitelist (png, jpg, jpeg, pdf, doc, docx); secure_filename; 16 MB request limit	“Invalid document format. Allowed: PNG, JPG, PDF, DOC, DOCX.”
Decide a leave only once	Route checks status = pending, then UPDATE and INSERT in one transaction	“Application has already been approved/rejected.”
No duplicate attendance	UNIQUE (session_id, student_id) with ON DUPLICATE KEY UPDATE so a re-save corrects instead of duplicating	Same record updated; marked_by and marked_at refreshed
Valid attendance status	Any value other than present, absent, leave defaults to present; ENUM at database level	Domain enforced by ENUM('present','absent','leave')
Faculty act only on own sections	Route compares session / section faculty_id with the logged-in faculty_id	“Unauthorized: You can only record attendance for your own sessions.”
Enrolment integrity and capacity	UNIQUE (student_id, section_id); route checks max_capacity; only admin may modify	“Cannot enroll student: Section has reached its maximum capacity”
Unique master data	Routes pre-check duplicates; UNIQUE constraints back them up	“Course code … already exists.”; “A student with roll number … already exists.”
Shortage alert logic	After attendance save, percentage is recomputed; alert only if at least 2 counted classes and below threshold; resolved when threshold is met	attendance_alerts row status active / resolved

Two points about transaction scope should be noted. First, adding a student or faculty member inserts into users and then into the profile table with two separate execute calls, so the pair is not a single transaction; a failure of the second insert would leave an unused user row. Second, the alert refresh runs after the attendance transaction has committed and is therefore a separate unit of work. Both are listed as improvements in Section 4.1.

3.6 Reports and Database Operations

The Reports Hub provides eight analytical reports (Table 14). Every report is generated by a SQL query, can be filtered where indicated, and can be downloaded as CSV through the same route with export=csv. The attendance calculation is Attendance % = Present ÷ (Present + Absent + Leave) × 100, so a leave mark counts as a class in the denominator but not in the numerator.

Table 14. Reports and Database Operations

Report / Operation	SQL / Database Concept Used	Application Output
1. Student-wise Attendance %	SELECT from view v_student_course_attendance; filters on department and course	Per student and course: counted, present, absent, leave, percentage
2. Course-wise Summary	Multi-table LEFT JOINs; COUNT(DISTINCT), SUM(CASE), ROUND, NULLIF; GROUP BY course	Sections, sessions held, total present marks, average attendance %

Report / Operation	SQL / Database Concept Used	Application Output
3. Attendance Shortage Roster	Joins; GROUP BY with HAVING (at least 2 counted classes and percentage below threshold); GREATEST / CEIL for shortfall	Students below threshold with classes needed to recover
4. Leave History and Audit	Joins of applications, students, types, approvals, users, faculty; COALESCE for reviewer	Application, type, dates, status, reviewer, notes
5. Pending Leave Queue	Filtered SELECT (status = pending) with joins; ordered by applied_at	Requests awaiting review
6. Faculty Allocation Matrix	LEFT JOINs with a derived-table subquery counting enrolled students per section; COUNT(DISTINCT)	Sections, courses, sessions taught and student load per faculty
7. Makeup Participation	LEFT JOIN makeup_attendance; COUNT and SUM(CASE)	Makeup sessions with students attended
8. Monthly Attendance Trends	DATE_FORMAT, GROUP BY month, aggregate counts	Sessions and attendance rate per month
Dashboards	Aggregate COUNT / SUM queries; JSON data for Chart.js	Cards and charts for each role
Attendance marking sheet	Correlated scalar subquery for each student's overall attendance %	Roster with current percentage beside each student

A representative query is Report 3 (shortage roster), shown here in abbreviated form; the threshold is passed as a bound parameter.

```
SELECT s.roll_number, s.full_name, c.course_code, sec.section_name,
       COUNT(a.attendance_id) AS total_counted,
       SUM(CASE WHEN a.status = 'present' THEN 1 ELSE 0 END) AS attended,
       ROUND(SUM(CASE WHEN a.status='present' THEN 1 ELSE 0 END)
             / NULLIF(COUNT(a.attendance_id),0) * 100, 2) AS current_pct
FROM enrollments e
JOIN students s ON e.student_id = s.student_id
JOIN sections sec ON e.section_id = sec.section_id
JOIN courses c ON sec.course_id = c.course_id
JOIN class_sessions cs ON sec.section_id = cs.section_id
JOIN attendance a ON cs.session_id = a.session_id AND a.student_id =
s.student_id
WHERE e.status = 'enrolled'
GROUP BY e.enrollment_id, s.roll_number, s.full_name, c.course_code,
sec.section_name
HAVING COUNT(a.attendance_id) >= 2 AND current_pct < %s
ORDER BY current_pct ASC;
```

The leave decision is the principal project-specific transaction. Both statements below are executed through execute_transaction, so the application's status and the audit row are saved together or not at all.

```
UPDATE leave_applications SET status = %s WHERE application_id = %s;
INSERT INTO leave_approvals (application_id, reviewed_by, decision,
decision_notes, reviewed_at)
VALUES (%s, %s, %s, %s, CURRENT_TIMESTAMP);
-- COMMIT on success; ROLLBACK if either statement raises an error
```

3.7 Application Screens

Figures 2 to 5 present the screens that demonstrate data entry and update (attendance marking), a project-specific transaction (leave review), search and display (student attendance summary)

and a report output with CSV export. The screenshots must be captured from the running application with the supplied demonstration data.

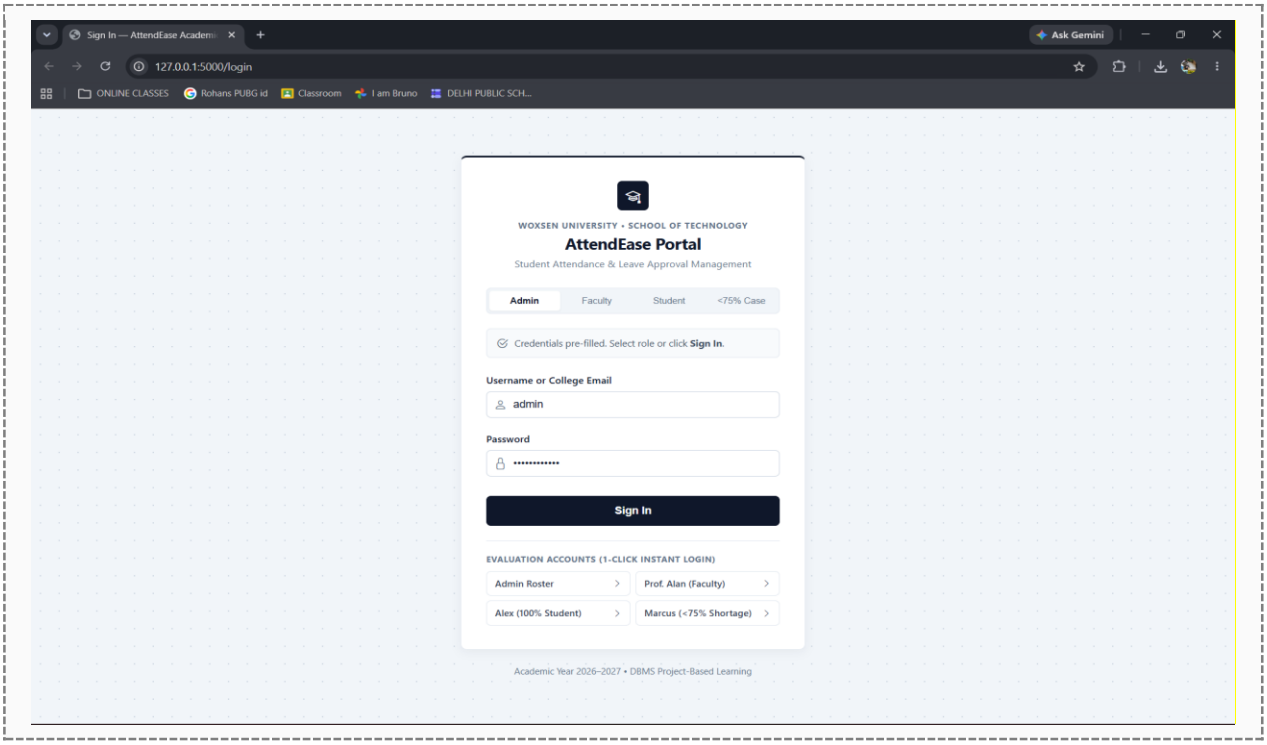


Figure 2. Attendance Marking Screen (Data Entry and Update Transaction)

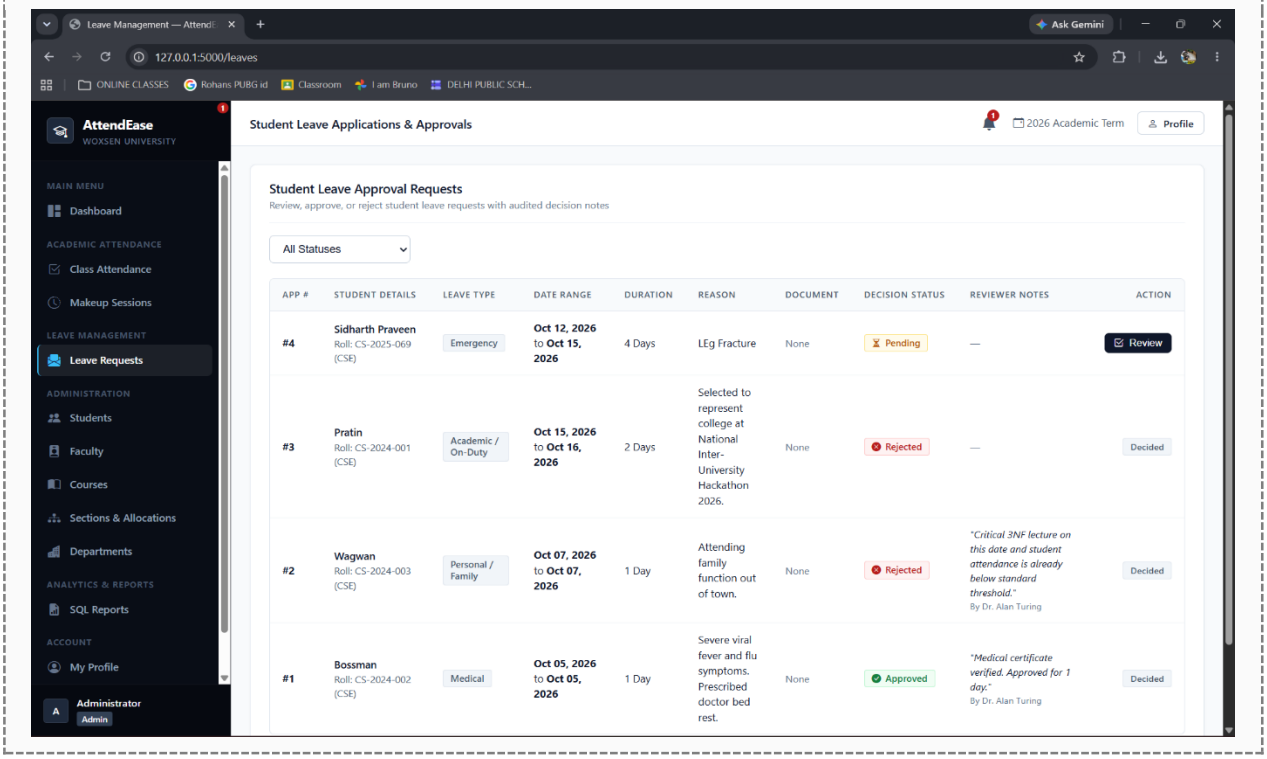


Figure 3. Leave Application and Review Screen (Approval Transaction)

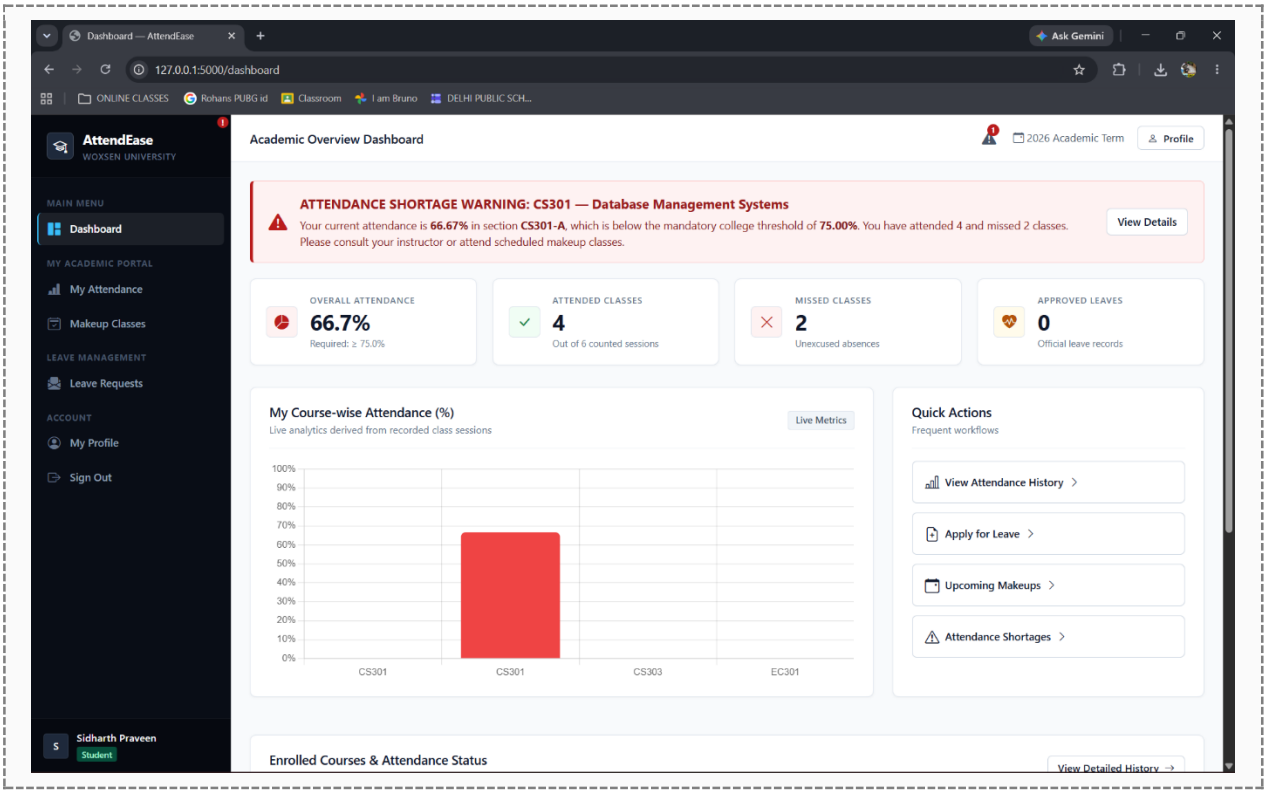


Figure 4. Student Attendance Summary with Shortage Alert (Search / Display)

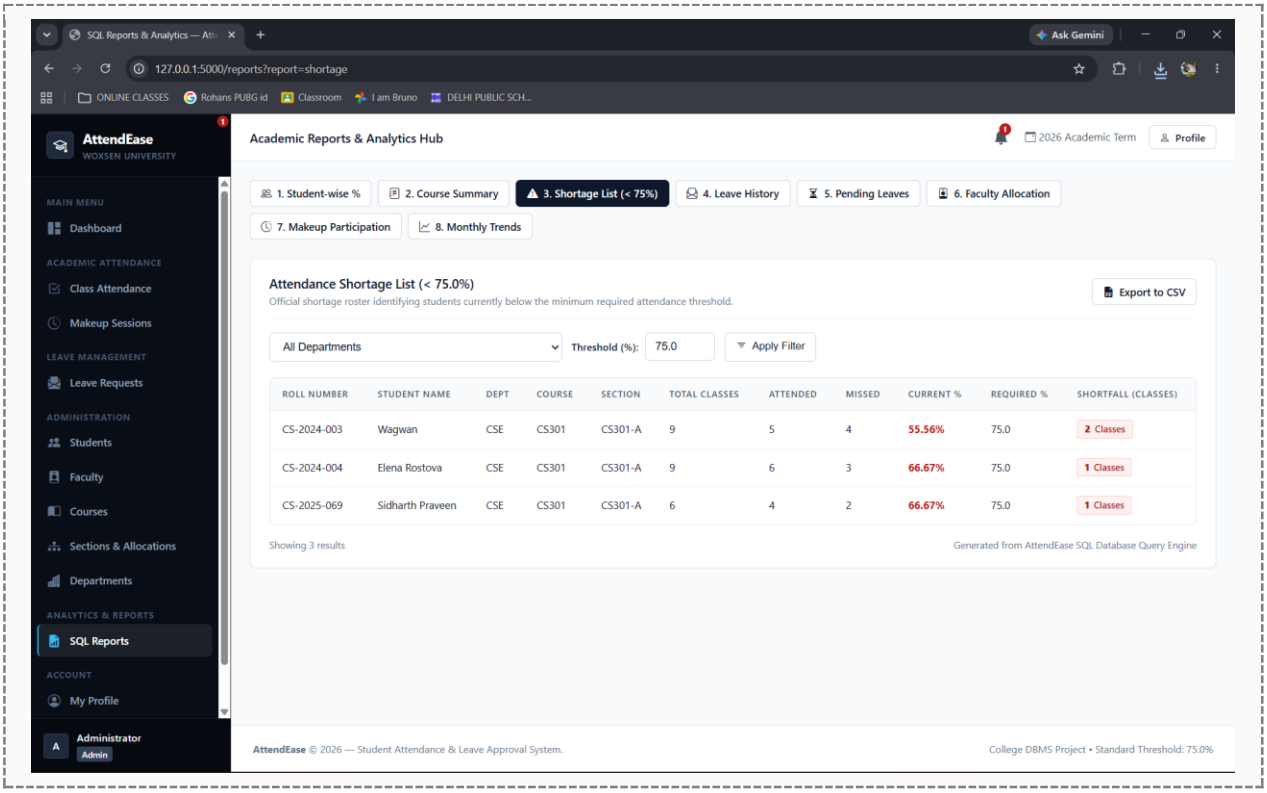


Figure 5. Reports Hub with CSV Export (Report Output)

3.8 Testing and Results

Table 15 lists positive and negative test cases derived from the implemented rules and the supplied seed data. The environment used to prepare this report had neither a MySQL server nor network access, so the application could not be executed there. The expected results are therefore taken from the source code and seed data, and the actual results and status are left as pending until each case is run against a live database. The repository also contains test_attendease.py with nine unit-test methods (password hashing, percentage arithmetic, shortage rule, date validation, redirection of unauthenticated users, role restrictions, login page and CSV response); these exercise logic with literal values and route redirections rather than the database, and were likewise not executed for this report.

Table 15. Test Cases

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Login (positive)	Username alex_rivera with the demo password	Login succeeds; redirected to the student dashboard; USER_LOGIN row added to audit_logs	To be recorded after execution	Pending
TC-02	Login (negative)	Valid username, wrong password	Login refused with “Invalid username/email or password.”; no session created	To be recorded after execution	Pending
TC-03	RBAC	Logged in as student, open /faculty	Redirected to the dashboard with an “Access denied” message	To be recorded after execution	Pending
TC-04	Mark attendance	Faculty prof_alan marks session 3 of CS301-A: Alex present, Priya leave, Marcus absent	Rows saved in one transaction; a repeat save updates the same rows (no duplicates)	To be recorded after execution	Pending
TC-05	Percentage & alert	View Marcus Vance’s CS301 attendance with seed data	4 present of 8 counted = 50.00%; an active alert exists because 50 < 75	To be recorded after execution	Pending
TC-06	Duplicate enrolment	Admin enrols a student already enrolled in a section	Rejected with “Student is already enrolled in this section.”; row count unchanged	To be recorded after execution	Pending
TC-07	Leave dates (negative)	Student submits leave with end date earlier than start date	Rejected with the end-date validation message; no row inserted	To be recorded after execution	Pending
TC-08	Leave approval	Faculty approves pending application 3	Status becomes approved and one leave_approvals row is added together; both or neither	To be recorded after execution	Pending
TC-09	Re-review	Open the review action again for an already decided application	Refused with “Application has already been …”; no new approval row	To be recorded after execution	Pending
TC-10	CSV export	Admin opens Reports (Student-wise) with export=csv	Response of type text/csv with a header row and one row per enrolment	To be recorded after execution	Pending

Key results and observations. A review of the source code against schema.sql shows that the application uses the designed database for all its data: every base table is referenced by application queries, relationships are navigated with joins on the declared foreign keys, the

unique and check constraints back up the route-level validation, and the multi-statement operations (attendance save, leave decision, makeup attendance) are transactional. Conformance of runtime behaviour with the expected results in Table 15 remains to be confirmed by executing the test cases.

CHAPTER 4
CONCLUSION

This project addressed the fragmented handling of attendance and leave in a college by designing and implementing AttendEase, a database-driven application. An ER model was converted into a seventeen-table relational schema with primary, foreign, unique, ENUM and CHECK constraints and two views, and the design was analysed for functional dependencies and normalization. Three documented exceptions to strict 3NF were retained deliberately for convenience in reporting.

On top of this design, a Python Flask application with a MySQL back end provides role-based workspaces for students, faculty and administrators. It records attendance in three states, computes percentages and shortage alerts, processes leave requests with an audit trail inside a transaction, schedules makeup sessions, and produces eight SQL reports with CSV export. The work gave practical experience of connecting ER modelling, constraints, joins, aggregation, views and transaction control to a working application.

4.1 Limitations and Future Enhancements

- An approved leave is not applied automatically to attendance; faculty must mark the affected sessions as “leave” manually. Linking approval to the attendance rows (for example inside the approval transaction) would remove this step.
- Any faculty member or administrator can decide any pending leave request, because the review route checks the role but not whether the student studies under that faculty member. Restricting review to the faculty of the student’s sections would tighten the rule.
- Adding a user and its student or faculty profile, and the alert refresh after attendance, are separate commits. Wrapping each pair in one transaction would make them atomic.
- Forms do not use CSRF tokens, error messages may display raw database exception text, uploaded leave documents are served to any logged-in user who knows the file name, and the application falls back to a default secret key and debug mode if the environment file is not configured. These should be hardened before real deployment.
- The shortage report uses the threshold from configuration or the request, whereas the alert logic reads min_attendance_threshold from system_settings; the two sources should be unified. The view v_section_summary and the leave_types and system_settings tables currently have no management screen or consumer.

- Possible extensions are stored procedures or triggers for percentage and alert maintenance, e-mail notifications, timetable integration, and automated database-level tests that run against a test MySQL instance.

REFERENCES

[1] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Hoboken, NJ, USA: Pearson, 2016.

[2] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[3] Oracle Corporation, MySQL 8.0 Reference Manual. Accessed:14 October 2026 . [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/

[4] Pallets, Flask Documentation, ver. 3.1. Accessed: 14 October 2026. [Online]. Available: https://flask.palletsprojects.com/

[5] Pallets, Werkzeug Documentation. Accessed: 14 October 2026. [Online]. Available: https://werkzeug.palletsprojects.com/

[6] Chart.js Contributors, Chart.js Documentation, ver. 4.4. Accessed: 14 October 2026. [Online]. Available: https://www.chartjs.org/docs/

[7] A. Singh, “AttendEase – Student Attendance & Leave Approval Management System (DBMS_PBL),” GitHub repository. Accessed: 14 October 2026. [Online]. Available: https://github.com/atharva-singh-07/DBMS_PBL

APPENDICES

APPENDIX A - COMPLETE DATABASE SCRIPT

The complete database scripts are part of the repository [7] and are submitted with this report:

File	Lines	Content
schema.sql	396	Creates database attendease_db; drops and recreates 17 tables with all constraints and indexes; defines views v_student_course_attendance and v_section_summary.
seed.sql	179	Sample data for all 17 tables (default demo password hash for all demo accounts).
init_db.py	211	Helper that connects to MySQL, executes schema.sql and seed.sql, and verifies the tables.

The definition of the main reporting view is reproduced below.

```
CREATE OR REPLACE VIEW v_student_course_attendance AS
SELECT
    e.enrollment_id,
    e.student_id,
    s.roll_number,
    s.full_name AS student_name,
    s.dept_id,
    d.dept_code,
    d.dept_name,
    e.section_id,
    sec.section_name,
    sec.academic_term,
    sec.academic_year,
    c.course_id,
    c.course_code,
    c.course_title,
    c.credits,
    f.faculty_id,
    f.full_name AS faculty_name,
    COUNT(cs.session_id) AS total_sessions_conducted,
    COUNT(a.attendance_id) AS total_counted_classes,
    SUM(CASE WHEN a.status = 'present' THEN 1 ELSE 0 END) AS present_count,
    SUM(CASE WHEN a.status = 'absent' THEN 1 ELSE 0 END) AS absent_count,
    SUM(CASE WHEN a.status = 'leave' THEN 1 ELSE 0 END) AS leave_count,
    COALESCE(
        ROUND(
            (SUM(CASE WHEN a.status = 'present' THEN 1 ELSE 0 END) /
             NULLIF(COUNT(a.attendance_id), 0)) * 100,
            2
        ),
        0.00
    ) AS attendance_percentage
FROM enrollments e
JOIN students s ON e.student_id = s.student_id
JOIN departments d ON s.dept_id = d.dept_id
JOIN sections sec ON e.section_id = sec.section_id
JOIN courses c ON sec.course_id = c.course_id
JOIN faculty f ON sec.faculty_id = f.faculty_id
LEFT JOIN class_sessions cs ON cs.section_id = sec.section_id
LEFT JOIN attendance a ON a.session_id = cs.session_id AND a.student_id =
s.student_id
WHERE e.status = 'enrolled'
GROUP BY
```

```
    e.enrollment_id, e.student_id, s.roll_number, s.full_name, s.dept_id,
    d.dept_code, d.dept_name, e.section_id, sec.section_name,
sec.academic_term,
    sec.academic_year, c.course_id, c.course_code, c.course_title, c.credits,
    f.faculty_id, f.full_name;
```

APPENDIX B - APPLICATION SOURCE CODE

The complete source code is available at https://github.com/atvaholic/DBMS_PBL [7]. The principal folders and files are:

Path	Description
app.py, config.py	Flask application factory, blueprint registration, template filters, error handlers; configuration loaded from environment variables.
database/connection.py	Connection pool and the helper functions query_all, query_one, execute, execute_transaction, log_audit.
routes/	auth, dashboard, students, faculty, courses, attendance, leaves, makeup and reports blueprints.
templates/, static/	Eighteen Jinja2 page templates; CSS design system and JavaScript.
schema.sql, seed.sql, init_db.py	Database definition, demonstration data and initialisation script.
test_attendease.py	Unit-test script (nine test methods).
requirements.txt, .env.example	Pinned dependencies and a template for environment settings (no real credentials).

Installation and execution

```
git clone https://github.com/atvaholic/DBMS_PBL.git
cd DBMS_PBL
python -m pip install -r requirements.txt
copy .env.example .env          (then edit .env: set DB_PASSWORD and
SECRET_KEY)
python init_db.py --password <your_mysql_password>
python app.py                   (opens at http://127.0.0.1:5000)
```

Demonstration accounts created by seed.sql are admin, prof_alan, prof_grace, alex_rivera, priya_patel and marcus_vance; the shared demonstration password is documented in the repository README.